الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري
Arab Academy for Science, Technology & Maritime Transport
College of Computing and Information Technology

CS460/CAI3105

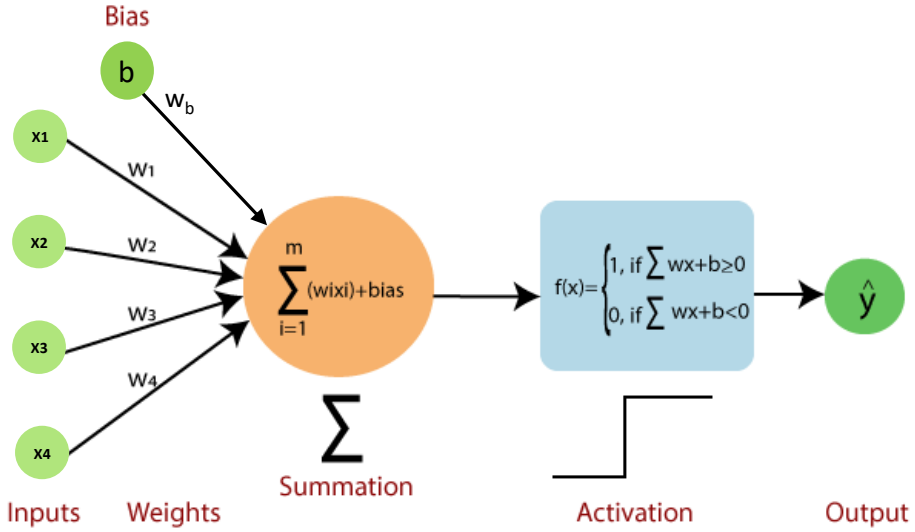
DEEP LEARNING

LECTURE-2

Multi Layer Perceptron (MLP)
Backpropagation

Single-Layer Perceptron (SLP)

- A single-layer artificial neural network, also called a Single-Layer Perceptron (SLP), has a single layer of nodes. Each node in the single layer connects directly to an input variable and contributes to an output variable.
- SLPs have just one layer of active units. Inputs connect directly to the outputs through a single layer of weights. The outputs do not interact, so a network with N outputs can be treated as N separate single-output networks.



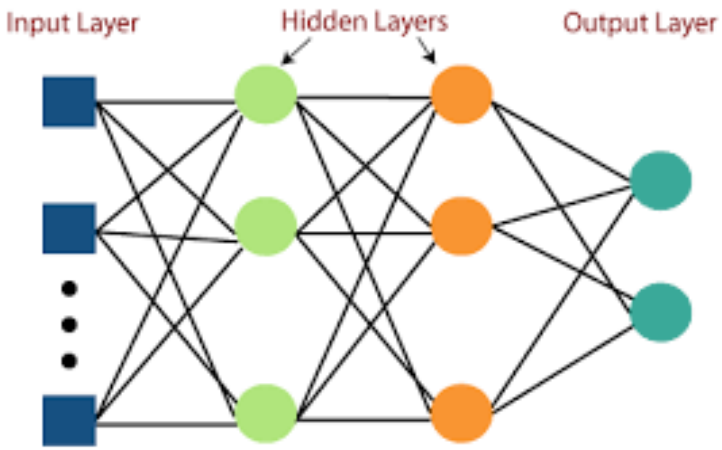
Multi-Layer Perceptron (MLP)

- Multi Layer Perceptron (MLP) is an Artificial Neural Network consists of at least three layers of nodes: an input layer, a hidden layer, and an output layer.

- Except for the input nodes, in MLP each node is a neuron that uses a non-linear activation function.

- MLP has an input layer (consists of input nodes) that connects to the input variables (features), one or more (intermediate interior layers) hidden layers that are not directly observable from the systems inputs and outputs, and an output layer (consists of input nodes) that produces the output variables (prediction or classification decision).

- MLP utilizes a supervised learning technique called (backpropagation) for training in addition to the non-linear activation function, which distinguishes MLP from a linear perceptron and also enables it to classify data that is not linearly separable.



Why using MLP?

- The SLP ANN can only be used to represent linearly separable functions. This means very simple classification problems with two classes that can be easily separated by a line. So, in similar simple cases, perhaps a single layer network would be sufficient.

- Most problems in real-life are not linearly separable.

- MLP can be used to represent convex regions. This means that in effect, they can learn to draw shapes around examples in some high-dimensional space that can separate and classify them, overcoming the limitation of linear separability.

- MLP, with at least one hidden layer of sufficient hidden neurons, which represents a feedforward network with a linear output layer and an activation function (ex: sigmoid activation function), can approximate any measurable function with any desired non-zero amount of error.

Backpropagation

Backpropagation → the weights of the network connections are repeatedly adjusted to minimize the difference between the actual output vector of the net and the desired output vector.

- Backpropagation aims to **minimize the loss (total error)** by adjusting the network's weights and biases.
- The cost function gradients determine the level of adjustment with respect to parameters like activation function, weights, bias, etc.
- The idea behind backpropagation is that it's unknown what the hidden units ought to do, but how fast the error changes as a hidden activity changes, can be computed.
- Each hidden activity can affect many output units and can therefore have many separate effects on the error.
- These effects must be combined in order to compute error derivatives for all the hidden units efficiently.
- Once the error derivatives for the hidden activities is computed, its easy to get the error derivatives for the weights going into a hidden unit.

Backpropagation steps
(gradient calculation: single epoch, single pattern)

1. **Forward pass:**
   - Input of each pattern of the input data is propagated through the network, layer by layer, for computing the output predictions.
   - The hidden and output layers have 2 functions at each node - linear transformation (to calculate the weighted sum of (input values * corresponding weights) & activation function (to generate the output of that node).

2. **Loss calculation:**
   - After forward passing is done for each pattern, the output predictions are compared to target values to compute the loss.

3. **Backpropagation:**
   - Used to calculate the gradients of the loss function with respect to the weights of the network.
   - It has 2 steps - backward pass (that calculates the δ) & gradient calculation (that calculates Δw).

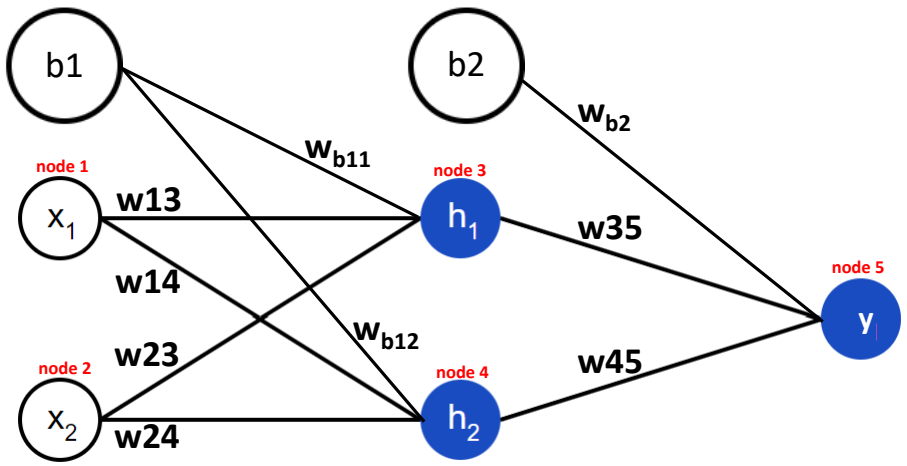
4. **Weight update:**
   - After calculating the gradients, the total weights of the network are updated ($w_{new}=w_{old}+\Delta w$)to minimize the loss function.

For each pattern

- Forward prop
  - Calculate net_j and o_j for all neurons (except input layer and bias neurons)
  - Calculate specific error (for single pattern)
- Back prop
  - Calculate δ_j for all neurons (except input layer and bias neurons)
  - Calculate $\Delta w_{i,j}$ for all variable weights including bias weights
  - $w_{i,j} := w_{i,j} + \Delta w_{i,j}$

MLP Backpropagation (Example-1)

- Consider the following ANN with an input layer consists of 2 perceptrons corresponding to 2 inputs (feature values) **x1** (node1) and **x2** (node2), a single hidden layer consists of 2 neurons **h1** (node3) and **h2** (node4), and an output layer with 1 neuron **y** (node5).
- Assume an input value of 2 patterns (2 feature vectors), with the corresponding values of initial random weight of the first pattern and the initial random weight value of (bias) in input and hidden layers ($\mathbf{b_1}$=1) and ($\mathbf{b_2}$=1), as shown in the following table.
- Given the shown target classification value **t** for both patterns, show the steps of the forward pass and backpropagation assuming that the **sigmoid** activation function [$f(a)=1/1+e^{-a}$] is applied in neurons of both hidden and output layers and given that the learning rate (η)= 0.4



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	-	-	-	-	-	-	-		-	1

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

➢ **1) Forward pass:**

- Weighted sum at node3 (h1):

s3=(x1*w13)+(x2*w23)+$\mathbf{w_{b11}}$ = **1.6**

- Observation at node3 (h1):

o3=1/(1+e^{-s3})= **0.832**

- Weighted sum at node4 (h2):

s4=(x1*w14)+(x2*w24)+$\mathbf{w_{b12}}$ = **1**

- Observation at node4 (h2):

o4=1/(1+e^{-s4})= **0.731**

- Weighted sum at node5 (output y):

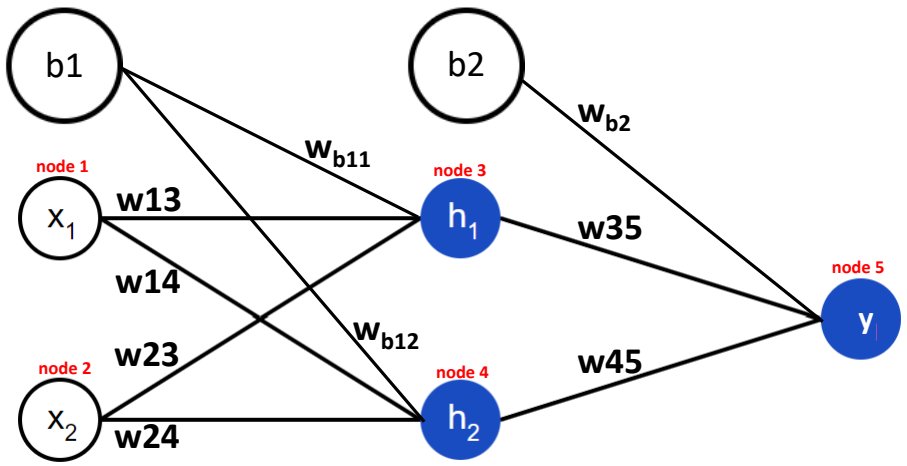
s5=(o3*w35)+(o4*w45)+$\mathbf{w_{b2}}$ =**1.151**

- Observation at node5 (Y):

y=1/(1+e^{-s5})= **0.760**

➢ **2) Error (Loss) calculation** of pattern 1:

E: Mean Square Error of patter 1 =**1/2 * (t-y)²** = **0.289**



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	-	-	-	-	-	-	-		-	1

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

➢ **3) Backpropagation:**

→ Calculating δ for **sigmoid** activation function:

Nodes of output layer: $\delta_y = y*(1-y)*(t-y)$

Nodes of hidden layer: $\delta_{h(i)} = o_{h(i)} *(1-o_{h(i)})* (w_{h(i)\,j} *\delta_j)$

$\delta 5 = y*(1-y)*(t-y)$

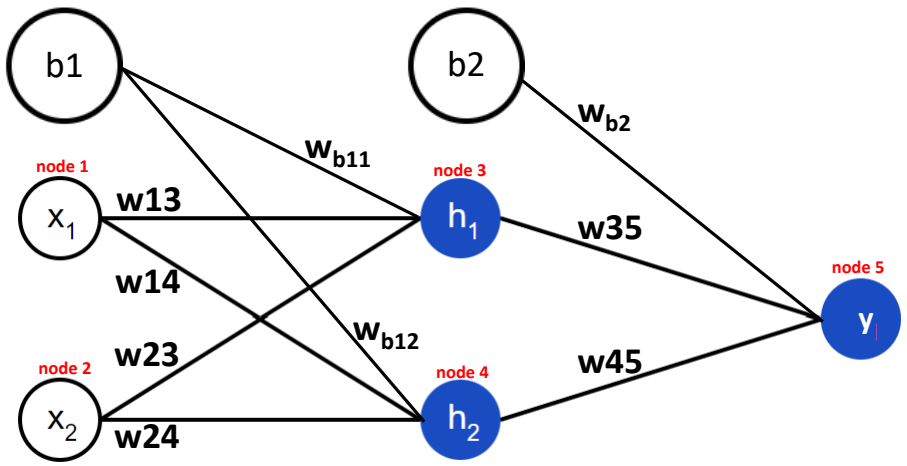
$\delta 4 = o4 *(1-o4)* (w45 * \delta 5)$

$\delta 3 = o3 *(1-o3)* (w35 * \delta 5)$

$\delta 5 = -0.139$

$\delta 4 = -0.008$

$\delta 3 = -0.019$



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	-	-	-	-	-	-	-		-	1

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

- **4) Weight update:**

→ Calculating Δw, (η= 0.4):

$\Delta wij = \eta * oi * \delta j$

→ Calculating the new weights:

$w_{new} = w_{old} + \Delta w$

$w35_{new}=w35_{old} + \Delta w_{35} = 1 + (0.4*0.832*-0.139)$

$w35_{new}=w35_{old} + \Delta w_{35} = 1 + (-0.046) = 0.954$

$w45_{new}=w45_{old} + \Delta w_{45} = 0.3 + (-0.041) = 0.259$

$w_{b2new}=w_{b2old} + \Delta w_{b2} = 0.1 + (-0.055) = 0.045$

$w13_{new}=w13_{old} + \Delta w_{13} = 0.5 + (-0.016) = 0.484$

$w14_{new}=w14_{old} + \Delta w_{14} = 0.1 + (-0.007) = 0.093$

$w23_{new}=w23_{old} + \Delta w_{23} = 0.1 + (-0.023) = 0.077$

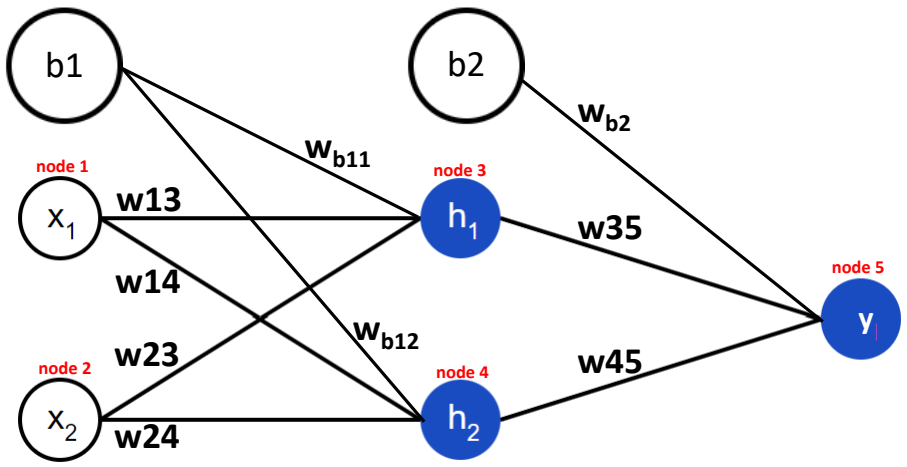
$w24_{new}=w24_{old} + \Delta w_{24} = 0.2 + (-0.010) = 0.190$

$w_{b11new}=w_{b1old} + \Delta w_{b11} = 0.3 + (-0.008) = 0.292$

$w_{b12new}=w_{b1old} + \Delta w_{b12} = 0.2 + (-0.003) = 0.197$

End of Pattern:1

→ Start Pattern:2 with the new weights



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	0.484	0.093	0.077	0.190	0.954	0.259	0.292	0.197	0.045	1

Epoch: 1
Pattern:2 x1=8, x2=5, t=1

➢ **Forward pass:**

- Weighted sum at node3 (h1):

s3=(x1*w13)+(x2*w23)+$\mathbf{w_{b11}}$ **= …**

- Observation at node3 (h1):

o3=1/(1+e^{-s3})= **…**

- Weighted sum at node4 (h2):

s4=(x1*w14)+(x2*w24)+$\mathbf{w_{b12}}$ **= …**

- Observation at node4 (h2):

o4=1/(1+e^{-s4})= **…**

- Weighted sum at node5 (output y):

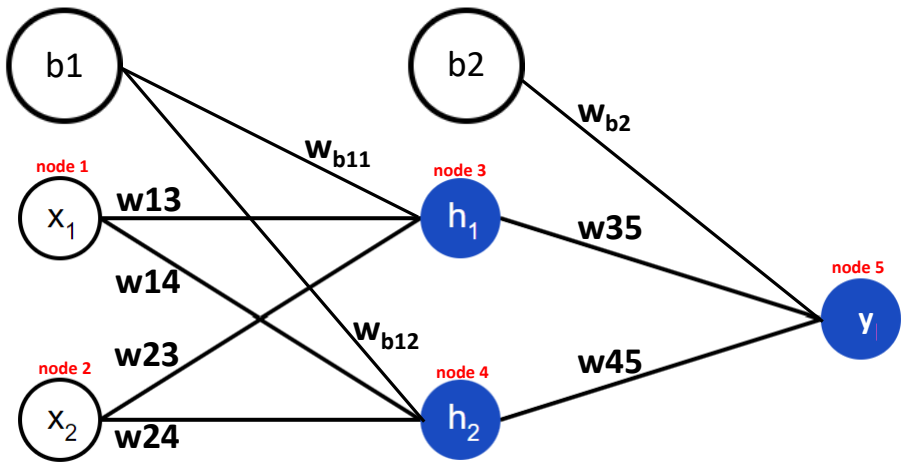
s5=(o3*w35)+(o4*w45)+$\mathbf{w_{b2}}$ = **…**

- Observation at node5 (Y):

y=1/(1+e^{-s5})= **…**

➢ **Error (Loss) calculation** of pattern2:

E: Mean Square Error of patter1=**1/2 * (t-y)²** = **…**



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	0.484	0.093	0.077	0.190	0.954	0.259	0.292	0.197	0.045	1

Epoch: 1
Pattern:2 x1=8, x2=5, t=1

- **Weight update:**

→ Calculating Δw, (η= 0.4):

$\Delta wij = \eta * oi * \delta j$

→ Calculating the new weights:

$w_{new} = w_{old} + \Delta w$

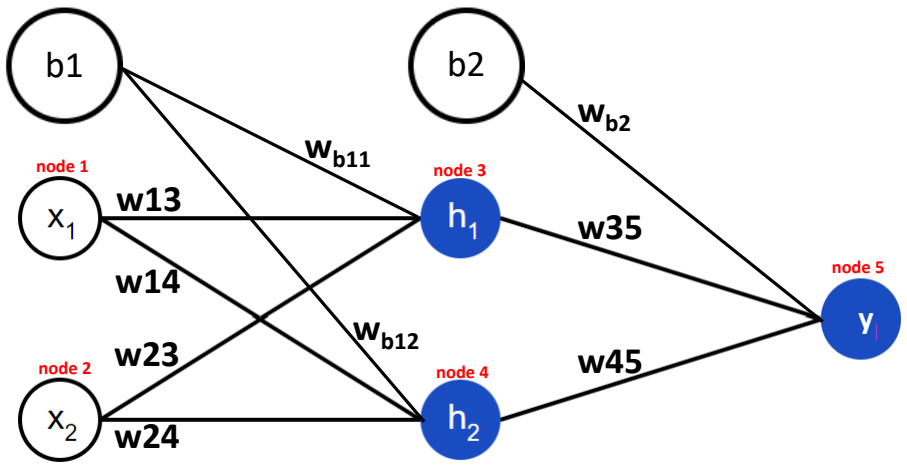
$w35_{new}=w35_{old} + \Delta w_{35} = \ldots$
$w45_{new}=w45_{old} + \Delta w_{45} = \ldots$
$w_{b2new}=w_{b2old} + \Delta w_{b2} = \ldots$
$w13_{new}=w13_{old} + \Delta w_{13} = \ldots$
$w14_{new}=w14_{old} + \Delta w_{14} = \ldots$
$w23_{new}=w23_{old} + \Delta w_{23} = \ldots$
$w24_{new}=w24_{old} + \Delta w_{24} = \ldots$
$w_{b11new}=w_{b1old} + \Delta w_{b11} = \ldots$
$w_{b12new}=w_{b1old} + \Delta w_{b12} = \ldots$

- **Total Loss (Epoch:1)** = $\Sigma_{i=1 \rightarrow \#patterns}$ error:pattern$_i$

If Total error <= tolerance (If given): Then stop training

If epoch number = max number of epochs (if given): Then stop training

Otherwise, run another epoch using last weights



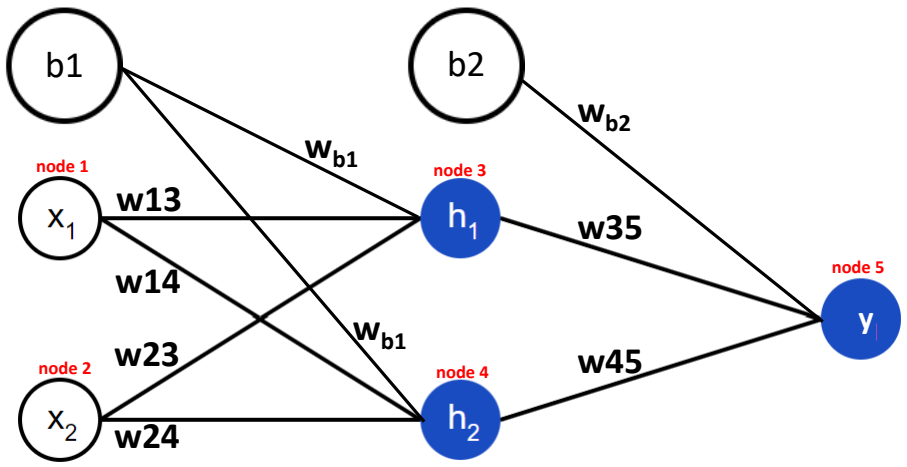
x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
2	3	0.5	0.1	0.1	0.2	1	0.3	0.3	0.2	0.1	0
8	5	0.484	0.093	0.077	0.190	0.954	0.259	0.292	0.197	0.045	1
		-	-	-	-	-	-	-	-	-	

MLP Backpropagation (Example-2)

(a single epoch: X-OR problem)

- Consider the following ANN with an input layer consists of 2 perceptrons corresponding to 2 inputs (feature values) **x1** (node1) and **x2** (node2) represent the input patterns to an X-OR circuit, a single hidden layer consists of 2 neurons **h1** (node3) and **h2** (node4), and an output layer with 1 neuron **y** (node5).
- Assume that the values of initial randomly generated weights are as shown in the following table:

Show the steps of the forward pass and backpropagation training process of (The First Epoch), assuming that the sigmoid activation function ([$f(a)=1/1+e^{-a}$] is applied in neurons of both hidden and output layers and given that the learning rate (η)= 0.3



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
0	0	0.3	-1.2	-0.2	0.1	0.4	-0.1	0.8	0.6	-0.1	0
0	1	-	-	-	-	-	-	-	-	-	1
1	0	-	-	-	-	-	-	-	-	-	1
1	1	-	-	-	-	-	-	-	-	-	0

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

➢ **Forward pass:**

- Weighted sum at node3 (h1):

s3=(x1*w13)+(x2*w23)+$\mathbf{w_{b11}}$ = **0.8**

- Observation at node3 (h1):

o3=1/(1+e^{-s3})= **0.69**

- Weighted sum at node4 (h2):

s4=(x1*w14)+(x2*w24)+$\mathbf{w_{b12}}$ = **0.6**

- Observation at node4 (h2):

o4=1/(1+e^{-s4})= **0.646**

- Weighted sum at node5 (output y):

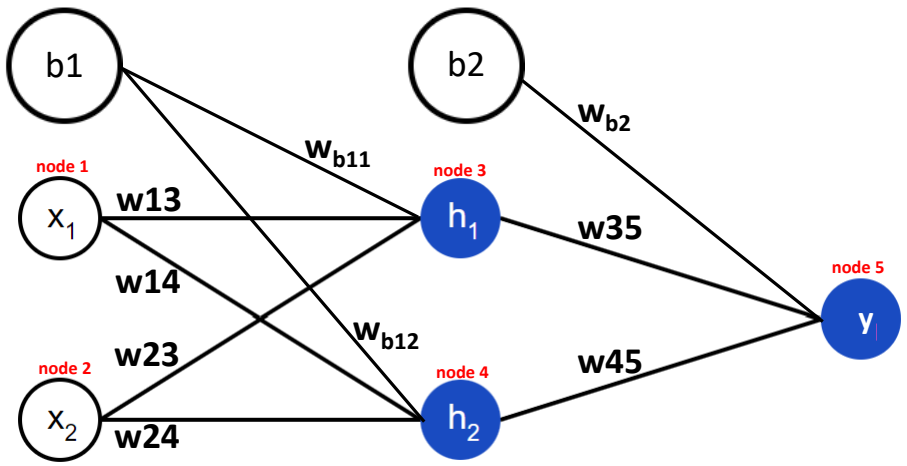
s5=(o3*w35)+(o4*w45)+$\mathbf{w_{b2}}$ = **0.111**

- Observation at node5 (y):

y=1/(1+e^{-s5})= **0.528**

➢ **Error (Loss) calculation** of pattern1:

E: Mean Square Error of patter1=**1/2 * (t-y)^2** = **0.139**



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
0	0	0.3	-1.2	-0.2	0.1	0.4	-0.1	0.8	0.6	-0.1	0
0	1	-	-	-	-	-	-	-	-	-	1
1	0	-	-	-	-	-	-	-	-	-	1
1	1	-	-	-	-	-	-	-	-	-	0

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

- **Backpropagation:**

→ Calculating δ for **sigmoid** activation function:

Nodes of output layer: $\delta_y = y*(1-y)*(t-y)$

Nodes of hidden layer: $\delta_{h(i)} = o_{h(i)} *(1-o_{h(i)})* (w_{h(i)\,j} *\delta_j)$

$\delta 5 = y*(1-y)*(t-y)$

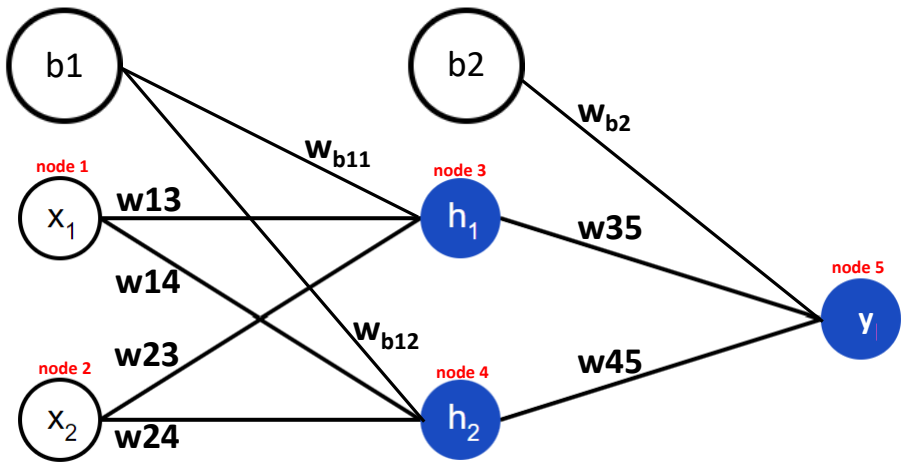
$\delta 4 = o4 *(1-o4)* (w45 * \delta 5)$

$\delta 3 = o3 *(1-o3)* (w35 * \delta 5)$

$\delta 5 = -0.132$

$\delta 4 = 0.003$

$\delta 3 = -0.011$



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
0	0	0.3	-1.2	-0.2	0.1	0.4	-0.1	0.8	0.6	-0.1	0
0	1	-	-	-	-	-	-	-	-	-	1
1	0	-	-	-	-	-	-	-	-	-	1
1	1	-	-	-	-	-	-	-	-	-	0

Epoch: 1
Pattern:1 x1=2, x2=3, t=0

- **Weight update:**

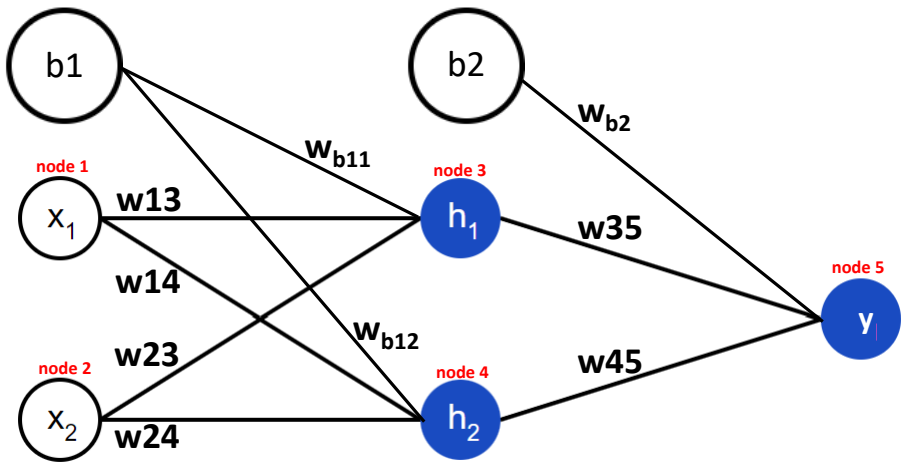
→ Calculating Δw, (η= 0.3):

$\Delta wij = \eta * oi * \delta j$

→ Calculating the new weights:

$w_{new} = w_{old} + \Delta w$

$w35_{new}=w35_{old} + \Delta w_{35} = \ldots$
$w45_{new}=w45_{old} + \Delta w_{45} = \ldots$
$w_{b2new}=w_{b2old} + \Delta w_{b2} = \ldots$
$w13_{new}=w13_{old} + \Delta w_{13} = \ldots$
$w14_{new}=w14_{old} + \Delta w_{14} = \ldots$
$w23_{new}=w23_{old} + \Delta w_{23} = \ldots$
$w24_{new}=w24_{old} + \Delta w_{24} = \ldots$
$w_{b11new}=w_{b1old} + \Delta w_{b11} = \ldots$
$w_{b12new}=w_{b1old} + \Delta w_{b12} = \ldots$



x1	x2	w13	w14	w23	w24	w35	w45	w_{b11}	w_{b12}	w_{b2}	t
0	0	0.3	-1.2	-0.2	0.1	0.4	-0.1	0.8	0.6	-0.1	0
0	1	**0.3**	**-1.2**	**-0.2**	**0.1**	**0.373**	-0.125	0.792	0.601	-0.139	1
1	0	-	-	-	-	-	-	-	-	-	1
1	1	-	-	-	-	-	-	-	-	-	0

End of Pattern:1

→ Start Pattern:2 with the new weights